

Digital Twin Architecture and Vehicle Mathematics for the ITQ Robby Platform

Aditya

May 8, 2026

Contents

Abstract

This document describes the current digital twin pipeline for the ITQ Robby mobile robot. The original mechanical platform was designed as a four-wheel-drive, four-wheel-steer robot. For the current software stage, however, the platform is intentionally simplified to an Ackermann-style vehicle by locking the rear steering joints at zero and commanding only the front steering joints. The present system contains a ROS 2 robot description, Gazebo Sim integration, joint-level control through `ros2_control`, wheel-based odometry, IMU preprocessing, and Extended Kalman Filter (EKF) fusion through the `robot_localization` package. This report summarizes the mathematical model used in the current software, the role of each ROS package, the frame definitions, the estimation pipeline, and the current limitations.

Chapter 1

Introduction

1.1 Motivation

The main objective of the current work is to create a simulation and estimation pipeline that is as close as possible to a real robotic deployment. Rather than relying on perfect world-state information from the simulator, the software stack is designed to infer the robot state from simulated sensor information, in the same way that a physical robot would rely on wheel encoders, steering states, and IMU measurements.

1.2 Current Simplification Strategy

Although the long-term platform goal is a steer-drive robot with richer motion modes, the current software uses an Ackermann-style approximation in order to develop the digital twin incrementally. The simplification is defined by the following assumptions:

- the rear steering joints are locked at zero,
- the front steering joints define the instantaneous turning geometry,
- all four wheels remain driven,
- state estimation is based on wheel motion and IMU data rather than simulator ground truth.

Chapter 2

Robot Frames and Coordinate Conventions

2.1 Frame Definitions

The current robot model uses the following principal frames:

- **base_link**: abstract root frame of the robot,
- **chassis_link**: physical chassis body under **base_link**,
- **imu_link**: IMU mounting frame attached to the chassis,
- **odom**: locally consistent odometry frame used for motion tracking,
- **map**: reserved global frame for future localization correction.

The transform chain used in the present system is

$$\text{odom} \rightarrow \text{base_link} \rightarrow \text{chassis_link} \rightarrow \text{joint/link tree.} \quad (2.1)$$

2.2 Interpretation of the Frames

The **base_link** frame is the logical robot body reference used by controllers, odometry, and localization. The **chassis_link** is the actual rigid body that contains the mesh, inertia, and collision geometry. This distinction is useful because it avoids assigning root-link inertia directly to **base_link** while preserving a clean TF tree for estimation and control.

Chapter 3

Vehicle Geometry

3.1 Key Parameters

The current software uses the following geometric parameters:

$$L = \text{wheelbase} \approx 0.406 \text{ m}, \quad (3.1)$$

$$W = \text{track width} \approx 0.33112 \text{ m}, \quad (3.2)$$

$$r_w = \text{wheel radius} = 0.052 \text{ m}. \quad (3.3)$$

These quantities are defined in the xacro description and reused by the command and odometry nodes.

3.2 Ackermann Simplification

For the current stage, the steerable wheels are reduced to the two front wheels. The rear steering joints remain at zero:

$$\delta_{rl} = 0, \quad \delta_{rr} = 0. \quad (3.4)$$

The front steering joints are denoted by

$$\delta_{fl}, \quad \delta_{fr}. \quad (3.5)$$

Chapter 4

Control Mathematics

4.1 Command Interface

The robot receives a body-frame velocity command

$$\mathbf{u} = \begin{bmatrix} v_x \\ \omega_z \end{bmatrix}, \quad (4.1)$$

where v_x is the commanded longitudinal velocity and ω_z is the commanded yaw rate.

In the current Ackermann simplification, lateral velocity v_y is ignored.

4.2 Instantaneous Turning Radius

For a non-zero yaw command, the nominal turning radius is

$$R = \left| \frac{v_x}{\omega_z} \right|. \quad (4.2)$$

The inner and outer rear radii are then

$$R_{\text{in}} = R - \frac{W}{2}, \quad (4.3)$$

$$R_{\text{out}} = R + \frac{W}{2}. \quad (4.4)$$

4.3 Steering Angles

The corresponding front steering angles are

$$\delta_{\text{in}} = \tan^{-1} \left(\frac{L}{R_{\text{in}}} \right), \quad (4.5)$$

$$\delta_{\text{out}} = \tan^{-1} \left(\frac{L}{R_{\text{out}}} \right). \quad (4.6)$$

For a left turn ($\omega_z > 0$):

$$\delta_{fl} = \delta_{\text{in}}, \quad (4.7)$$

$$\delta_{fr} = \delta_{\text{out}}. \quad (4.8)$$

For a right turn ($\omega_z < 0$):

$$\delta_{fl} = -\delta_{\text{out}}, \quad (4.9)$$

$$\delta_{fr} = -\delta_{\text{in}}. \quad (4.10)$$

4.4 Wheel Linear Speeds

The rear wheel linear speeds are

$$v_{rl} = s |\omega_z| R_l, \quad (4.11)$$

$$v_{rr} = s |\omega_z| R_r, \quad (4.12)$$

where $s = \text{sign}(v_x)$, and R_l, R_r are the left and right turning radii after the left/right assignment has been resolved.

The front wheel linear speeds follow from the front wheel path radii:

$$v_{fl} = s |\omega_z| \sqrt{R_l^2 + L^2}, \quad (4.13)$$

$$v_{fr} = s |\omega_z| \sqrt{R_r^2 + L^2}. \quad (4.14)$$

Wheel angular velocities are obtained by

$$\dot{\phi}_i = \frac{v_i}{r_w}. \quad (4.15)$$

Chapter 5

Wheel Odometry Model

5.1 Motivation

The current estimation architecture intentionally avoids using simulator ground truth as the primary source of robot pose. Instead, the simulator provides sensor-like feedback through joint states and IMU messages. This allows the software stack to resemble the structure required on real hardware.

5.2 Encoder-Derived Wheel Travel

For each wheel, the incremental wheel travel is computed from the change in joint position:

$$\Delta s_i = \Delta \phi_i r_w. \quad (5.1)$$

Because the right-side wheel joints use mirrored joint axes in the URDF, sign correction is applied before odometry is computed. In software, this is represented by per-wheel multipliers:

$$\Delta s_i^{\text{corr}} = m_i \Delta s_i, \quad (5.2)$$

where $m_i \in \{-1, +1\}$ depends on the wheel joint orientation convention.

5.3 Current Ackermann Odometry Approximation

The current raw wheel odometry is not based on the full four-wheel swerve least-squares model during normal operation. Instead, it uses a rear-axle Ackermann approximation:

$$\Delta s_r = \frac{1}{2} (\Delta s_{rl}^{\text{corr}} + \Delta s_{rr}^{\text{corr}}), \quad (5.3)$$

$$\Delta \psi = \frac{\Delta s_{rr}^{\text{corr}} - \Delta s_{rl}^{\text{corr}}}{W}. \quad (5.4)$$

The body-frame incremental displacement is therefore approximated as

$$\Delta x_b = \Delta s_r, \quad (5.5)$$

$$\Delta y_b = 0, \quad (5.6)$$

$$\Delta \theta = \Delta \psi. \quad (5.7)$$

5.4 Pose Update

Let the previous robot pose in the odometry frame be (x_k, y_k, θ_k) . The next pose is computed using midpoint heading integration:

$$\theta_{k+1/2} = \theta_k + \frac{1}{2}\Delta\theta, \quad (5.8)$$

$$x_{k+1} = x_k + \cos(\theta_{k+1/2})\Delta x_b - \sin(\theta_{k+1/2})\Delta y_b, \quad (5.9)$$

$$y_{k+1} = y_k + \sin(\theta_{k+1/2})\Delta x_b + \cos(\theta_{k+1/2})\Delta y_b, \quad (5.10)$$

$$\theta_{k+1} = \theta_k + \Delta\theta. \quad (5.11)$$

5.5 Velocities

Given the sample interval Δt , the reported body velocities are

$$v_x = \frac{\Delta x_b}{\Delta t}, \quad (5.12)$$

$$v_y = \frac{\Delta y_b}{\Delta t}, \quad (5.13)$$

$$\omega_z = \frac{\Delta\theta}{\Delta t}. \quad (5.14)$$

Chapter 6

IMU Processing

6.1 Raw IMU Source

In simulation, the IMU is defined as a Gazebo sensor attached to `imu_link`. The IMU data is bridged from Gazebo to ROS 2 using `ros_gz_bridge`.

6.2 IMU Preprocessing

Before entering the EKF, the IMU message is normalized by a preprocessing node. The current preprocessing performs the following tasks:

- forces the ROS frame identifier to `imu_link`,
- fills orientation covariance if it is missing,
- fills angular-velocity covariance if it is missing,
- fills linear-acceleration covariance if it is missing.

This step is important because state estimation requires a consistent frame name and non-degenerate covariance matrices.

Chapter 7

Extended Kalman Filter Fusion

7.1 Implementation Strategy

The current EKF is not implemented manually inside this workspace. Instead, the system uses the ROS 2 package `robot_localization`, specifically the `ekf_node`. Therefore:

- the process and update logic are provided by the package,
- the workspace only provides measurement sources and configuration,
- the filter model is a generic rigid-body motion model rather than a custom tire-force vehicle model.

7.2 State Interpretation

The EKF internally estimates a general motion state including position, orientation, linear velocity, angular velocity, and linear acceleration. Conceptually, the state can be written as

$$\mathbf{x} = [x, y, z, \phi, \theta, \psi, v_x, v_y, v_z, \omega_x, \omega_y, \omega_z, a_x, a_y, a_z]^\top. \quad (7.1)$$

7.3 Measurements Used in the Current Pipeline

The current filter uses:

- raw wheel odometry from `/wheel/odom`,
- filtered IMU messages from `/imu/data/filtered`.

At the present tuning stage:

- wheel odometry contributes primarily longitudinal motion,
- IMU contributes heading and yaw rate,
- linear acceleration is currently not emphasized in the fusion.

7.4 Why the EKF is Needed

Wheel odometry alone accumulates drift and is sensitive to wheel slip, steering error, and imperfect kinematic assumptions. The IMU alone provides angular information and short-term inertial behavior, but not reliable position by itself. The EKF combines these partial sources into a smoother and more consistent local pose estimate.

Chapter 8

Software Architecture

8.1 Package-Level Structure

The current workspace is organized into four main packages:

Package	Role
<code>robby_description</code>	Robot geometry, xacro files, mesh resources, RViz launch, and the separation between the clean robot model and the Gazebo-specific overlay.
<code>robby_control</code>	Ackermann command conversion, wheel/joint command bridging, and raw wheel odometry estimation.
<code>robby_gazebo</code>	Gazebo world, <code>ros2_control</code> controllers, robot spawning, sensor bridging, and simulation bring-up.
<code>robby_localization</code>	IMU preprocessing and EKF configuration through <code>robot_localization</code> .

8.2 Main Executable Nodes

The present software pipeline includes the following main nodes:

Node	Function
<code>robot_state_publisher</code>	Publishes the TF tree from the robot description and current joint states.
<code>ackermann_cmd_node</code>	Converts <code>/cmd_vel</code> to steering angles and wheel velocity commands.
<code>ackermann_controller_bridge</code>	Converts joint-command style output into the command topics required by the ROS 2 controllers.
<code>joint_state_broadcaster</code>	Publishes <code>/joint_states</code> from the simulated hardware interfaces.
<code>swerve_odometry_node</code>	Computes raw wheel odometry, currently in Ackermann odometry mode.
<code>imu_preprocessor_node</code>	Normalizes IMU frame identifiers and covariance fields.
<code>ekf_filter_node</code>	Fuses wheel odometry and IMU data.

8.3 Current Topic Flow

The operational signal flow is:

```
/cmd_vel → ackermann_cmd_node  
          → /ackermann_cmd_joint_states  
          → ackermann_controller_bridge  
          → /steering_position_controller/commands, /wheel_velocity_controller/commands  
                                                    (8.1)
```

Meanwhile, the state-estimation side follows:

```
Gazebo joints → /joint_states → swerve_odometry_node → /wheel/odom,  
                                                    (8.2)
```

```
Gazebo IMU → /imu/data → imu_preprocessor_node → /imu/data/filtered  
                                                    (8.3)
```

```
(/wheel/odom, /imu/data/filtered) → ekf_filter_node → /odometry/filtered, odom → base_link  
                                                    (8.4)
```

Chapter 9

Why Gazebo Ground Truth is Not the Main State Source

9.1 Design Philosophy

Although Gazebo can provide perfect world-state information, the present digital twin intentionally does not use that as the primary feedback source for localization. The reason is that the long-term objective is not simply to animate a robot in simulation, but to create a software architecture that can later be transferred to real hardware.

9.2 Practical Interpretation

The simulator is therefore used primarily as a provider of:

- joint-state measurements,
- IMU measurements,
- simulation clock,
- controlled actuation through `ros2_control`.

This makes the pipeline closer to the physical robot, where no perfect world pose is available by default.

Chapter 10

Current Limitations

10.1 Kinematic Approximation

The platform is mechanically capable of richer motion than the present software uses. The current odometry and command stack assume Ackermann-like motion, which is a useful engineering simplification but not the final intended platform behavior.

10.2 Estimator Model

The current EKF is a generic package-level filter configuration. It is not a custom vehicle-state observer derived from a first-principles dynamic model. Therefore, it is best viewed as a robust engineering estimator rather than a final model-based observer.

10.3 Remaining Tuning Work

The following aspects still require refinement:

- consistency between simulated path radius and estimated path radius,
- IMU noise and covariance tuning,
- wheel odometry calibration,
- future transition back toward a full steer-drive or swerve-drive formulation.

Chapter 11

Conclusion

The current ITQ Robby software stack already forms a meaningful digital twin foundation. The robot description is separated from the Gazebo overlay, the command path is functional, the simulator uses `ros2_control`, and the state estimation pipeline relies on wheel motion and IMU data rather than direct simulator truth. At the present stage, the most accurate way to describe the pipeline is as a ROS 2 Ackermann-style estimation and control stack built on top of a mechanically richer steer-drive robot. This provides a practical base for future work on more advanced steering modes, improved localization, and eventual transfer to hardware.

Appendix: Current Files of Interest

- `src/robby_description/urdf/Robby_v1.core.xacro`
- `src/robby_description/urdf/Robby_v1.gazebo.xacro`
- `src/robby_description/urdf/Robby_v1.urdf.xacro`
- `src/robby_control/robby_control/ackermann_cmd_node.py`
- `src/robby_control/robby_control/swerve_odometry_node.py`
- `src/robby_control/config/swerve_odometry.yaml`
- `src/robby_gazebo/launch/sim_ackermann.launch.py`
- `src/robby_gazebo/config/ros2_controllers.yaml`
- `src/robby_localization/robby_localization/imu_preprocessor_node.py`
- `src/robby_localization/config/ekf.yaml`